

HACKER ATTACK

HIJACKING



10

LISTA DE FIGURAS

Figura 1 – TCP Session Initiation.....	7
Figura 2 – Processo do TCP Session Hijacking.....	7
Figura 3 – Estrutura de um pacote TCP.....	8
Figura 4 – Processo de estabelecimento de uma sessão.....	9
Figura 5 – Criação de um Dynamic Web Project.....	13
Figura 6 – Criação de um Dynamic Web Project – Parte 2.....	13
Figura 7 – Criação de um Dynamic Web Project – Parte 3.....	14
Figura 8 – Criação de um Dynamic Web Project – Parte 4.....	14
Figura 9 – Console de Desenvolvimento do Chrome.....	17
Figura 10 – Configuração do Cookie.....	17
Figura 11 – Exploit de acesso ao Cookies.....	18
Figura 12 – Configuração do Cookie com Blindagem.....	19
Figura 13 – Exploit de acesso ao Cookie (blindado).....	19

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Código da classe SmartMeter	15
Código-fonte 2 – Código da página login.html.....	16
Código-fonte 3 – Código da página sucesso.jsp.	16
Código-fonte 4 – Código Blindado para o SmartMeter	18

EMANIP

SUMÁRIO

SESSION HIJACKING	5
1 RECAPITULANDO	5
1.1 Introdução	5
2 TCP / UDP SESSION HIJACKING	6
3 HTTP SESSION HIJACKING	9
4 QUESTÕES DE SEGURANÇA NO USO DE COOKIES PARA SESSÃO	10
5 CONSIDERAÇÕES DE SEGURANÇA	11
6 LABORATÓRIO – SESSION HIJACKING	12
7 CONDUZINDO UM ATAQUE	20
CONCLUSÃO	20
REFERÊNCIAS	22

SESSION HIJACKING

1 RECAPITULANDO

Já vimos as técnicas relacionadas ao *SQL Injection*, tratando-se do uso de comandos SQL “deformados” que são passados como parâmetros em formulários e camadas de interface de modo que esses comandos consigam chegar ao banco de dados e sejam executados como comandos e não como informações a serem armazenadas ou consultadas.

1.1 Introdução

O *SQL Injection* é uma das classes de um grupo de injeções de código (*Code Injection*), que representa uma parcela significativa dos ataques feitos a sistemas corporativos e/ou governamentais, muitos deles sem as devidas blindagens ou proteções adequadas para esse tipo de invasão.

Dentre as proteções possíveis de serem utilizadas, envolvem desde o uso de detecções de comandos nos parâmetros provenientes de um formulário (via busca por padrões ou o uso de *regular expressions*), até a não utilização de determinadas partes relacionadas à Programação Dinâmica.

Exaurimos as formas principais de invasão com base em injeção para sistemas que não fazem a sanitização dos dados, nem o tratamento contra execução de código externo.

Existem outras formas de tentar ganhar o controle de um sistema sem necessariamente trabalhar na invasão de um servidor, em vez disso, a tentativa de invasão passa por meio de um usuário, que costuma ser o elo mais fraco do processo como um todo, pois poucos usuários tomam as medidas de segurança e realizam os procedimentos necessários.

Dentre essas técnicas focadas mais no lado do usuário, uma das mais utilizadas é o *Session Hijacking*, que nada mais é do que “roubar” ou “sequestrar” os dados de sessão de um usuário, mas o que seria essa tal de sessão?

A sessão é o conjunto de dados que são armazenados temporariamente quando um usuário está autenticado e/ou identificado em um sistema. Em sistemas web em suas camadas superiores, os dados de uma sessão costumam ficar em um *cookie file* (arquivo de cookie) dentro de uma pasta de armazenamento do navegador utilizado.

Quando um invasor faz uso do *Session Hijacking* para explorar essa vulnerabilidade, o intuito é obter informações enquanto elas transitam entre o servidor e a máquina do usuário ou no próprio dispositivo do usuário.

Existem duas formas normalmente utilizadas para realizar o *Session Hijacking*, o *TCP Session Hijacking* (ou *UDP Session Hijacking*) e o *HTTP Session Hijacking*, sendo a diferença entre eles o protocolo em que o ataque será realizado e também a camada em que o invasor atuará. Veremos como funcionam essas duas abordagens nos próximos itens.

2 TCP / UDP SESSION HIJACKING

O TCP Session Hijacking se utiliza dos momentos iniciais do estabelecimento de uma conexão TCP, “sniffando” os pacotes entre o usuário e o servidor até o momento em que a sessão é estabelecida, quando o invasor captura o estado da conexão para assumir o lugar do usuário.

Para entendermos detalhadamente o funcionamento desse formato de ataque, vamos rever como funciona inicialmente o estabelecimento de uma conexão TCP e, depois, o momento no qual o invasor “assume” o lugar do usuário, sequestrando a sua sessão com o servidor.

Durante o estabelecimento de uma conexão TCP, o usuário e o servidor realizam um processo chamado de *3-way handshake*, que consiste na troca de pacotes SYN (synchronize) e ACK (acknowledgement), assim como no estabelecimento do código de sequência para garantir a confiabilidade e o fluxo de ordenamento de pacotes, conforme pode ser observado na figura “TCP Session Initiation”.

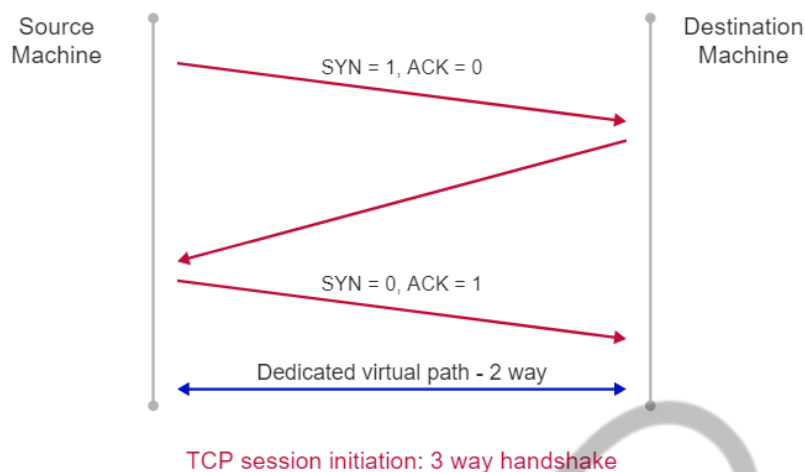


Figura 1 – TCP Session Initiation.
Fonte: Grey Campus (2018)

O invasor fica monitorando (sniffando) o estabelecimento da conexão, aguardando o momento em que o servidor enviará o código de sessão (*Session ID*). Com isso, o invasor começa a empregar esse código para utilizar os serviços no servidor como se fosse um usuário autenticado.

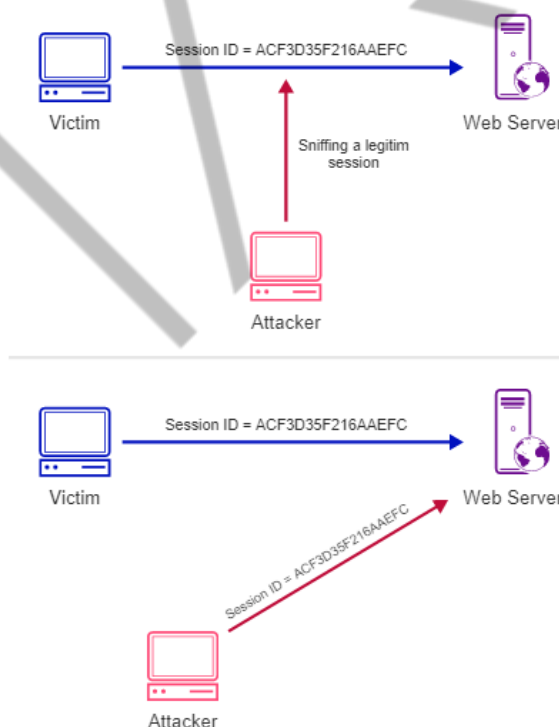


Figura 2 – Processo do TCP Session Hijacking.
Fonte: Grey Campus (2018)

Existem algumas proteções dentro do protocolo e do pacote TCP para evitar que esse tipo de ataque seja facilmente conduzido, como o uso de campos para porta de origem, número de sequência (como o TCP garante a ordenação dos pacotes, no caso de ocorrência de um número de sequência muito fora do comum, é um indício

de que um ataque esteja em andamento) e o *checksum* (que garante a integridade do conteúdo do pacote).

A estrutura de um pacote TCP e o nome dos principais campos constam na figura “Estrutura de um pacote TCP”.

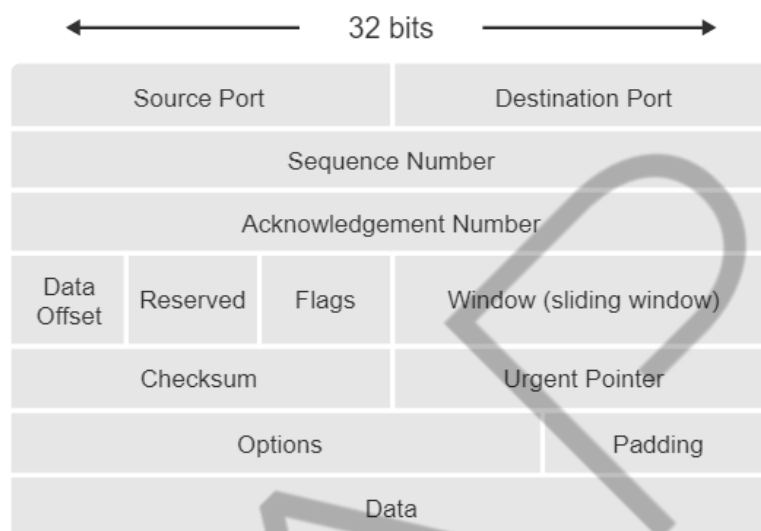


Figura 3 – Estrutura de um pacote TCP.
Fonte: Mullins (2001)

Existem variações desse tipo de ataque, utilizando-se de outros protocolos de rede, que valem a menção:

- IP Spoofing: o invasor envia pacotes IP como se fossem de um IP de origem legítima (imitando o dispositivo do usuário).
- Man in the Middle: utilizando ARP Spoofing, o invasor consegue atrelar o endereço MAC da sua placa de rede ao endereço IP do alvo, de maneira que todo tráfego de rede seja redirecionado inicialmente para a máquina do invasor (que redireciona novamente para o endereço de destino correto, por isso o nome do ataque).
- UDP Session Hijacking: é um ataque bem similar ao da versão do protocolo TCP, mas possuindo uma facilidade maior de implementação, devido ao fato do UDP não possuir várias das proteções do protocolo TCP, como o controle de fluxo de mensagens nem de integridade das informações do pacote.

Agora, veremos como um invasor pode fazer o *Session Hijacking* em camadas superiores (como na aplicação e na web), podendo fazer uso de *cookies* e do protocolo HTTP para sequestrar a sessão de um usuário.

3 HTTP SESSION HIJACKING

O HTTP Session Hijacking é uma variante de ataque que se utiliza das camadas superiores de rede para a sua execução, tendo como base o protocolo HTTP, podendo ser combinada com outras vulnerabilidades para conseguir um controle mais efetivo da sessão e do sistema.

Normalmente, o invasor se utiliza das informações armazenadas via *cookies* no dispositivo do usuário, que podem ser acessadas pelo invasor por diversos modos, dentre eles:

- Via XSS (Cross-site Scripting), por meio de um script injetado no client-side, de forma a exibir para o invasor as informações do *cookie*.
- Durante o processo de autenticação, quando o servidor utiliza a diretiva *set-cookie* para a criação do *cookie* no client-side.
- Conseguindo acesso ao browser ou sistema de arquivos diretamente, basta saber a localização do *cookie* em disco para a sua leitura.

Vale mencionar como funciona o processo de criação de um *cookie* durante o estabelecimento de uma sessão, que ocorre em três etapas usualmente, conforme ilustrado na figura “Processo de estabelecimento de uma sessão”.



Figura 4 – Processo de estabelecimento de uma sessão
Fonte: Microsoft (2012)

Inicialmente, o client-side faz uma requisição para o servidor, fornecendo os dados de usuário e senha (processo de autenticação). Supondo que o usuário tenha selecionado a opção de permanecer logado enquanto a sessão estiver ativa, quando o servidor verificar que as credenciais estão corretas, informa ao client-side o *HTTP Status Code 200 OK* e uma diretiva *set-cookie* que serve para avisar ao browser que um determinado conteúdo deve ser armazenado em um *cookie*.

Nesse caso, o *cookie* está armazenando a variável *session-id* com o valor 12345. Esse código de sessão permite que o client-side não precise ficar realizando o processo de autenticação a cada requisição futura ao servidor, evitando tráfego das credenciais repetidamente e desnecessariamente.

Após o estabelecimento do *cookie* com o *session-id*, o client-side enviará esse *cookie* para o servidor a cada requisição futura. Em um mundo ideal, apenas o browser do usuário e o servidor saberiam o valor do *session-id*, de maneira a manter uma comunicação segura, mas, infelizmente, esse não é um cenário factível, como veremos a seguir.

4 QUESTÕES DE SEGURANÇA NO USO DE COOKIES PARA SESSÃO

O uso de *cookies* para armazenamento de informações sensíveis (o *session-id*, por exemplo) necessita de uma configuração específica para evitar que o escopo de acesso a essa informação dê margem para que o invasor consiga acessar facilmente as informações contidas no *cookie*.

Essas configurações no *cookie* infelizmente não costumam vir habilitadas automaticamente por grande parte dos frameworks web, necessitando que o desenvolvedor e o analista de *security*, em especial, atentem com respeito a esse quesito.

Dois pontos de fácil verificação e que dão uma blindagem razoável para tentativas de invasão são o uso das flags **HttpOnly** e **Secure**, vale relembrar qual a finalidade dessas duas flags:

- **HttpOnly**: o *cookie* não é acessível internamente pelo browser do usuário, apenas pelo servidor do domínio ao qual o *cookie* está associado. Isso

impede as tentativas clássicas de obter o conteúdo do *cookie* via console de JavaScript contido em grande parte dos browsers modernos.

- **Secure:** o *cookie* apenas será transmitido por meio de canais seguros (criptografados), como em requisições HTTPS, por exemplo.]

5 CONSIDERAÇÕES DE SEGURANÇA

Como existem diversas formas de sequestrar uma sessão, tanto via protocolos de baixo-nível, como o TCP e o UDP, quanto em alto nível, como via HTTP, além de via mecanismos como *cookies*, não existe uma blindagem que funcionará contra todas as variantes, porém, podemos nos atentar a uma série de pontos capazes de dificultar a vida do invasor durante a execução de um ataque.

Segue uma lista (não exaustiva) de recomendações por meio das quais é possível evitar o surgimento de eventuais brechas de segurança:

- Utilize protocolos seguros (como o IPSec, HTTPS, SSL) para transmitir conteúdo sensível ou, se possível, qualquer tipo de informação. Evite o uso de protocolos que transmitam as informações em texto-puro (como o HTTP, Telnet e FTP).
- Encriptar ou evitar o uso de números sequenciais de *session-id* dificulta a vida do invasor que tenta prever quais são os números válidos de sessão do sistema. De preferência, utilize números aleatórios e encriptados.
- Faça uso de campos de *timeout* ou equivalentes, colocando uma data de validade para uma determinada sessão. No caso de *cookies*, seria utilizar o campo *expires*. Dessa forma, o invasor terá um tempo limitado para tentar conseguir acesso ao *session-id* de um usuário.
- Se o portal possui diversos níveis de acesso para diferentes usuários, utilize um *session-id* diferente para cada papel ou escopo de usuário. Isso limita o estrago que o invasor terá, caso consiga acesso a um *session-id*.
- Na infraestrutura de rede, evite o uso de bridges e de hubs, pois esses equipamentos não têm a inteligência necessária para detectar diversas

formas de *spoofing*. Dê preferência ao uso de roteadores e switches, no caso.

- Colocar monitoramento para ARP Spoofing / Poisoning.

Repare que as recomendações não se limitam apenas à parte de programação no sistema, mas também envolvem pontos na infraestrutura de rede e sistemas de monitoramento.

6 LABORATÓRIO – SESSION HIJACKING

Agora, vamos colocar a mão na massa em um exercício bem prático. As atividades são as seguintes:

- Subir um Apache Tomcat contendo uma tela e um Servlet de login para acesso a um SmartMeter.
- Analisaremos e faremos um diagnóstico no *cookie* gerado após um login bem-sucedido, assim como verificaremos se um invasor conseguiria facilmente obter a *session-id*.
- Encontradas as vulnerabilidades, aplicaremos as blindagens possíveis para tornar difícil a vida do invasor.

Antes de começar a configurar o projeto, é necessário realizar a instalação do Apache Tomcat, ele pode ser obtido no site: <<http://tomcat.apache.org/>>.

Recomenda-se instalar a versão 9.0 na versão Core. Em caso de dúvidas, existe um breve roteiro em: <https://tomcat.apache.org/tomcat-9.0-doc/setup.html>

Inicialmente, dentro do **Eclipse**, vamos criar um **Dynamic Web Project**, indo pelo caminho **File > New > Other...** . Depois, procure por **Dynamic Web Project**, como ilustrado na figura “Criação de um Dynamic Web Project”.

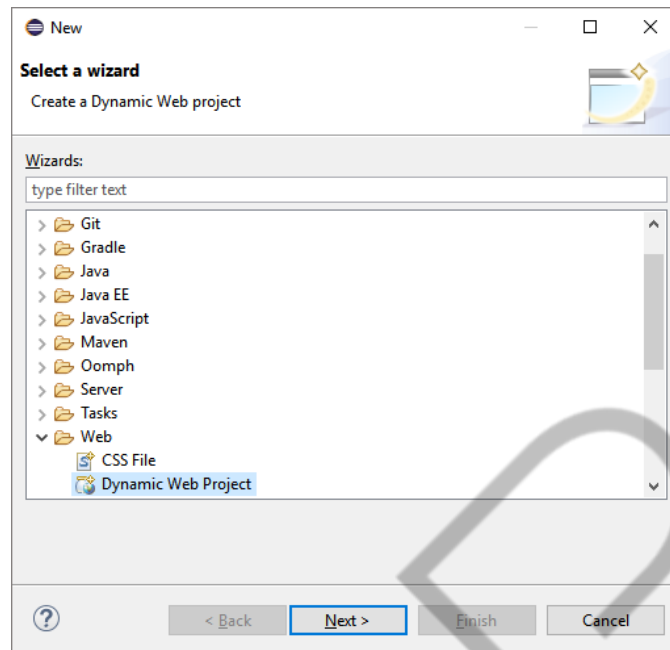


Figura 5 – Criação de um Dynamic Web Project
Fonte: Elaborado pelo autor (2019)

Nomeie o projeto como **SmartMeter**, depois, clique em **New Runtime...** (vamos configurar o Apache Tomcat agora), conforme a figura “Criação de um Dynamic Web Project – Parte 2”.

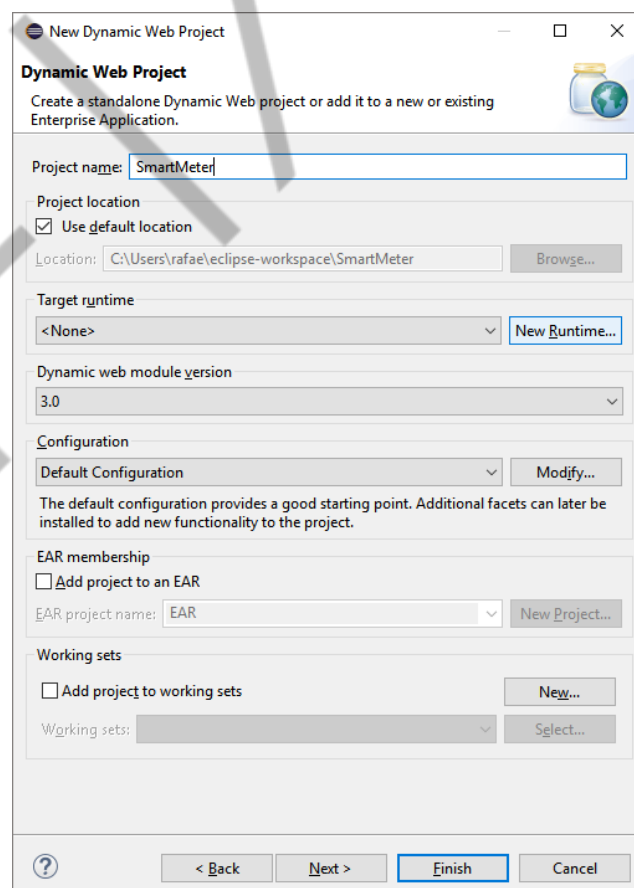


Figura 6 – Criação de um Dynamic Web Project – Parte 2

Fonte: Elaborado pelo autor (2019)

Procure por **Apache Tomcat 9.0** e coloque o local onde o Tomcat foi instalado, se o Eclipse não identificar automaticamente o local de instalação dele, clique em **Finish**, conforme a figura “Criação de um Dynamic Web Project – Parte 3”.

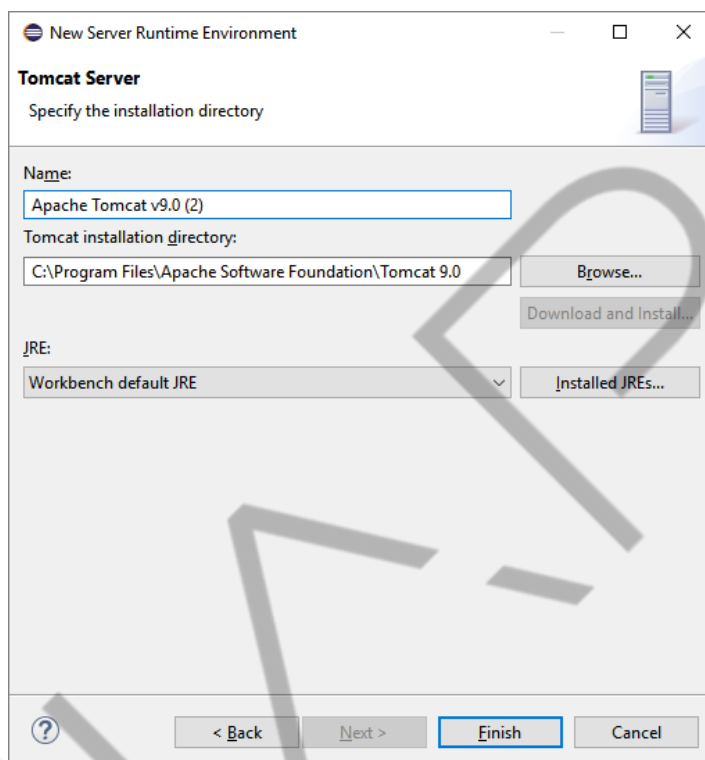


Figura 7 – Criação de um Dynamic Web Project – Parte 3
Fonte: Elaborado pelo autor (2019)

Com o Tomcat configurado, podemos prosseguir no assistente de criação de projeto, clique em **Next > Next** até chegar à seguinte tela, na qual a opção **Generate web.xml deployment descriptor** deverá ser selecionada. Depois, clique em **Finish**.

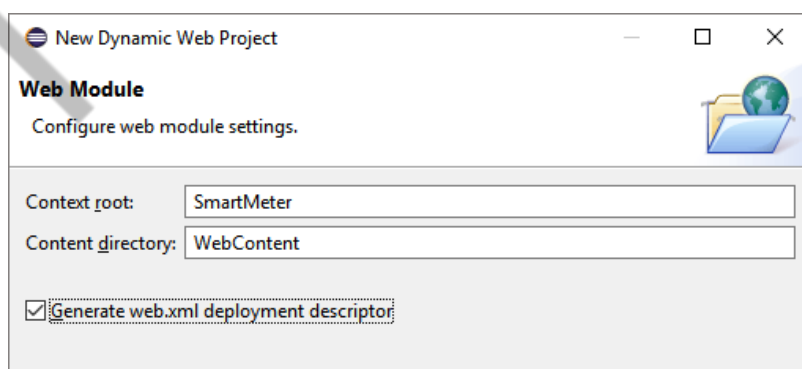


Figura 8 – Criação de um Dynamic Web Project – Parte 4
Fonte: Elaborado pelo autor (2019)

Após a etapa de criação do projeto, vamos criar os arquivos com o código referente ao serviço de login.

No menu esquerdo, com a listagem dos arquivos de projeto **SmartMeter**, entre em **Java Resources > src** e clique com o botão direito em **New > Class**. Coloque o nome da classe como **SmartMeter** e dê **Finish**.

Coloque o " Código da classe SmartMeter" na classe **SmartMeter**:

```
import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/SmartMeter")
public class SmartMeter extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        String usuario =
request.getParameter("usuario");
        String senha = request.getParameter("senha");

        if(usuario.equals("Rafael") &&
senha.equals("123456")) {
            String sessionid =
request.getSession().getId();
            String contextPath =
request.getContextPath();
            System.out.println(sessionid);
            response.setHeader("SET-COOKIE",
"JSESSIONID=" + sessionid
                + "; Path=" + contextPath);
            response.sendRedirect("sucesso.jsp");
        }
    }
}
```

Código-fonte 1 – Código da classe SmartMeter
Fonte: Elaborado pelo autor (2019)

Na pasta **WebContent**, crie um arquivo de nome **login.html** que conterá o "Código da página login.html":

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Login SmartMeter</title>
</head>
<body>
  <form
    action="http://localhost:8080/SmartMeter/SmartMeter"
    method="post">
    <label>Usuario:</label>
    <input type="text" name="usuario" />
    <label>Senha:</label>
    <input type="password" name="senha" />
    <button type="submit">Login</button>
  </form>
</body>
</html>
```

Código-fonte 2 – Código da página login.html.
Fonte: Elaborado pelo autor (2019)

Por fim, ainda na pasta **WebContent**, crie um arquivo de nome **sucesso.jsp**, contendo o “Código da página sucesso.jsp”:

```
<%@ page language="java" contentType="text/html;
charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Login!</title>
</head>
<body>
  <h1>O login foi realizado com sucesso!</h1>
</body>
</html>
```

Código-fonte 3 – Código da página sucesso.jsp.
Fonte: Elaborado pelo autor (2019)

Agora estamos preparados para ligar o servidor: vá até o arquivo **SmartMeter.java**, clique com o botão direito e vá a **Run As > Run on Server**. Após alguns segundos, o Tomcat estará no ar pela porta 8080 (padrão da aplicação).

O projeto está acessível localmente pelo seguinte link
<<http://localhost:8080/SmartMeter/login.html>>

Acesse esse link e entre com as credenciais **Rafael** (usuário) e **123456** (senha). Se tudo estiver configurado corretamente, a mensagem “O login foi realizado com sucesso!” aparecerá no browser.

Vamos analisar como o *session-id* está sendo armazenado nesse serviço. Para isso, entre no Console do Desenvolvedor (para esta etapa, recomenda-se fortemente o uso do Chrome ou do Firefox).

No Chrome, é possível acessar o console clicando com o botão direito na página e indo a **Inspect** ou utilizando a tecla de atalho **Ctrl + Shift + I**. Depois, entre na aba **Application**, conforme a figura “Console de Desenvolvimento do Chrome”.

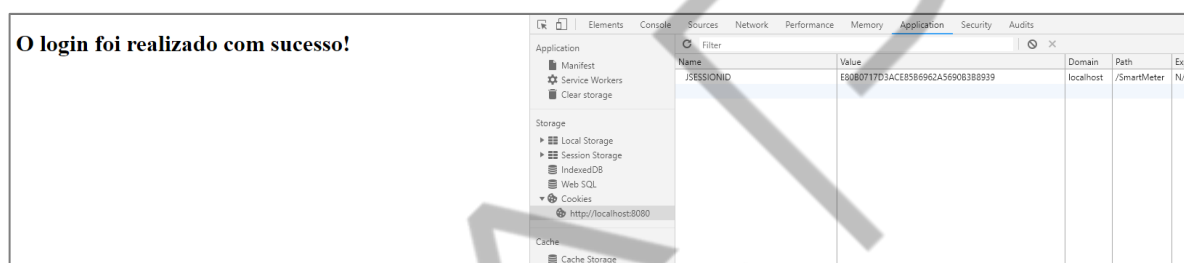


Figura 9 – Console de Desenvolvimento do Chrome.

Fonte: Elaborado pelo autor (2019).

Clique em **Cookies** e depois em **http://localhost:8080**, aparecerá na direita o conteúdo do cookie, com informações semelhantes à figura “Configuração do Cookie”:

Filter									
Name	Value	Domain	Path	Expir...	Size	HTTP	Secure	SameSite	
JSESSIONID	E80B0717D3ACE85B6962A5690B388939	localhost	/SmartMeter	N/A	42				

Figura 10 – Configuração do Cookie.

Fonte: Elaborado pelo autor (2018)

Cada linha dessa tela representa uma variável e um valor associado, assim como as flags de configuração do *cookie*. Repare que as flags **HTTP** (de HTTPOnly) e **Secure** não estão ativadas, ou seja, o invasor pode acessar o conteúdo desse *cookie* via XSS ou por alguma forma de acesso ao browser do usuário.

Para ilustrar como isso é fácil de verificar, na parte inferior dessa tela existe o console de comandos, digite **document.cookie** nele e dê **Enter**. Esse comando é, basicamente, um código JavaScript que o invasor poderia injetar de forma a capturar de maneira prática o conteúdo de um *cookie*, como apresentado na figura “Exploit de acesso ao Cookie”:

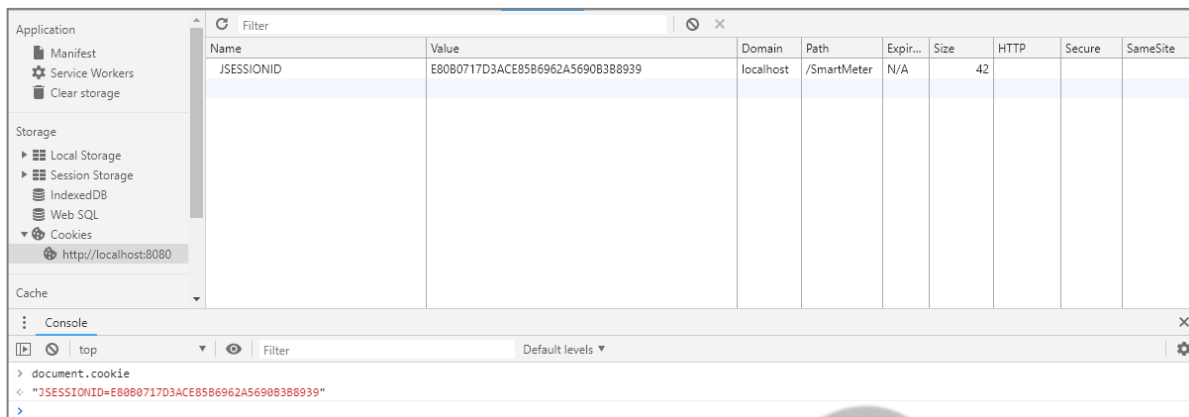


Figura 11 – Exploit de acesso ao Cookies
Fonte: Elaborado pelo autor (2019)

Precisamos evitar que isso ocorra, para tanto, vamos ativar a flag **HTTPOnly** e **Secure** no código da aplicação. Entre no arquivo **SmartMeter.java** e substitua o código existente pelo “Código Blindado para o SmartMeter”:

```

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/SmartMeter")
public class SmartMeter extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        String usuario = request.getParameter("usuario");
        String senha = request.getParameter("senha");

        if(usuario.equals("Rafael") &&
            senha.equals("123456")) {
            String sessionid =
                request.getSession().getId();
            String contextPath =
                request.getContextPath();
            System.out.println(sessionid);
            response.setHeader("SET-COOKIE",
                "JSESSIONID=" + sessionid
                    + "; Path=" + contextPath + ";
                HttpOnly; Secure");
            response.sendRedirect("sucesso.jsp");
        }
    }
}

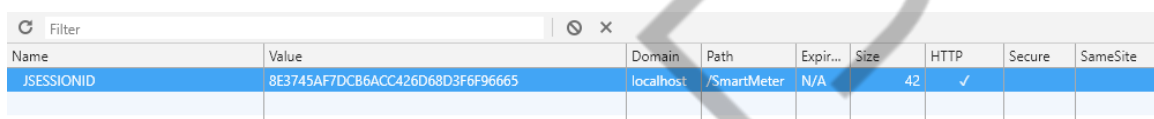
```

Código-fonte 4 – Código Blindado para o SmartMeter
Fonte: Elaborado pelo autor (2019)

Não será necessário reiniciar o sistema para execução do novo código, o Tomcat possui uma inteligência para detectar modificações no código e aplica a atualização on-the-fly.

Entre novamente em <http://localhost:8080/SmartMeter/login.html>, com as credenciais **Rafael** (usuário) e **123456** (senha). Se tudo estiver configurado corretamente, a mensagem “O login foi realizado com sucesso!” aparecerá no browser, como no caso anterior.

Abra o Console de Desenvolvimento do browser e repare que a flag **HTTP** está habilitada, conforme a figura “Configuração do Cookie com Blindagem”:



Name	Value	Domain	Path	Expir...	Size	HTTP	Secure	SameSite
JSESSIONID	8E3745AF7DCB6ACC426D68D3F6F96665	localhost	/SmartMeter	N/A	42	✓		

Figura 12 – Configuração do Cookie com Blindagem
Fonte: Elaborado pelo autor (2018)

Caso a flag **HTTP** não fique habilitada, tente clicar com o botão direito sobre <http://localhost:8080> e depois clique em **Clear**, e faça novamente o processo de login (isso limpará o cache e permitirá a gravação de um novo *cookie* com a configuração correta).

Essa nova versão blindada apenas acrescenta os termos **HttpOnly** e **Secure** no método **setHeader()**.

Vamos utilizar o comando **document.cookie** para ver o resultado:

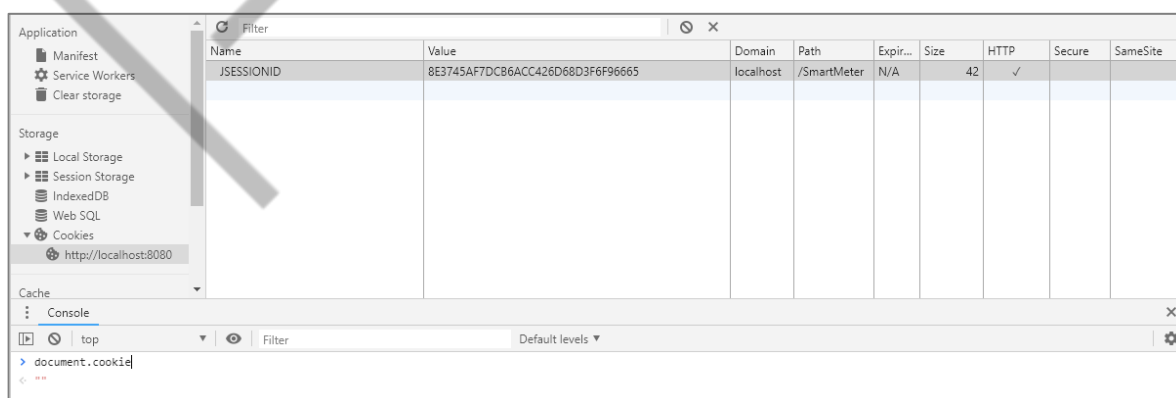


Figura 13 – Exploit de acesso ao Cookie (blindado)
Fonte: Elaborado pelo autor (2019)

Agora não é possível que um comando JavaScript local consiga acessar o conteúdo do *cookie*. Lembrando que a flag **HttpOnly** faz com que o *cookie* seja

acessado apenas pelo servidor e não localmente, essa blindagem não impede o invasor de tentar roubar a sessão via servidor.

Repare que a flag **Secure** não está habilitada mesmo com a sua configuração sendo repassada pelo header. Isso ocorre pelo fato de que o nosso acesso ao serviço está ocorrendo via HTTP e não via HTTPS. Ou seja, não adianta apenas setar a flag, nesse caso, é necessário que a aplicação se utilize do HTTPS também.

A configuração do HTTPS é um assunto para uma outra discussão, mas é válido pesquisar todo o processo relacionado ao fluxo de autenticação e estabelecimento de uma conexão segura.

Uma observação sobre a maneira de habilitar as flags do *cookie*: existem diversas maneiras de fazer essa configuração, isso depende fortemente do framework web utilizado, assim como da linguagem de programação utilizada (as abordagens podem mudar conforme o caso).

O método utilizado neste exercício prático foi o de acrescentar as flags diretamente no header do pacote de resposta. Essa é uma forma direta de configuração, mas funciona para boa parte dos frameworks web em Java.

7 CONDUZINDO UM ATAQUE

Existem várias formas de conduzir um *Session Hijacking*, utilizando desde protocolos de baixo nível (como o TCP / UDP), assim como em protocolos de alto nível (como o HTTP / HTTPS).

Foram apresentadas as formas de ataque via TCP e via HTTP (com a utilização de *cookies*), na qual o invasor pode ter acesso à variável de *session-id* (que armazena o código de sessão) a partir de um dispositivo do usuário.

CONCLUSÃO

Quando a implementação do *session-id* é com base no armazenamento via *cookies*, é possível blindar e dificultar eventuais tentativas de invasão fazendo uso das

flags **HttpOnly** e **Secure**, que limitam o acesso do *cookie* apenas ao servidor e a transferência do *cookie* ocorre apenas por meio de canais seguros.

Será que essa é a única forma de conseguir obter informações sensíveis por parte de um dispositivo de um usuário? Infelizmente, a resposta é não.

Há uma classe de ataques que é mais nociva do que o *Session Hijacking*, no sentido de que o usuário nem percebe que está em um ambiente inseguro e tendo os seus dados coletados.

EMANIP

REFERÊNCIAS

GREY CAMPUS. **Network or TCP Session Hijacking – Ethical Hacking**. Disponível em: <<https://www.greycampus.com/opencampus/ethical-hacking/network-or-tcp-session-hijacking>>. Acesso em: 04 jan. 2019.

MICROSOFT. **HTTP Cookies in ASP.NET Web API**. Disponível em: <<https://docs.microsoft.com/en-us/aspnet/web-api/overview/advanced/http-cookies>>. Acesso em: 04 jan. 2019.

MULLINS, Michael. **Exploring the anatomy of a data packet**. Disponível em: <<https://www.techrepublic.com/article/exploring-the-anatomy-of-a-data-packet/>>. Acesso em: 04 jan. 2019.